

Database Management System - 1

(Relational)

What is a Database?

A **Database** is an organized collection of **structured data or information** that is stored electronically in a computer system. It is designed to store data in a way that allows users to **easily access, manage, update, and retrieve information** whenever needed.

Instead of storing information randomly, databases organize data in a **systematic structure**, making it efficient to search, update, and maintain large amounts of information.

Example

Think of a **library**:

- Books are arranged by **topic, author, and title**.
- A librarian can quickly locate any book using this organization.

Similarly, a **database organizes data in a structured way** so that computers and users can find information quickly.

Characteristics of a Database

- Stores **large volumes of data**
- Maintains **organized structure**
- Allows **easy retrieval of information**
- Supports **multiple users accessing data simultaneously**

Why Do We Need Databases?

Databases are essential for managing large amounts of data efficiently. Without databases, handling and maintaining data would be extremely difficult.

Main Reasons for Using Databases

• Efficient Data Storage

Databases allow organizations to store large amounts of data in an organized way.

• Fast Data Retrieval

Users can quickly search and retrieve specific information using queries.

• Avoid Data Duplication

Databases help prevent storing the same data multiple times, reducing redundancy.

- **Data Integrity and Accuracy**

Rules and constraints ensure that stored data remains **correct and consistent**.

- **Multi-user Access**

Multiple users can access and work with the same database at the same time without conflicts.

Example

An **online shopping website** stores:

- Customer information
- Product details
- Order history
- Payment records

All this information is stored and managed efficiently using databases.

What is DBMS?

A **Database Management System (DBMS)** is software that allows users and applications to **interact with a database**.

It acts as an interface between the **database and users**, enabling them to create, modify, and manage data in an organized way.

Functions of a DBMS

- **Data Storage** – Stores large amounts of structured data
- **Data Retrieval** – Allows users to search and retrieve data quickly
- **Security** – Controls who can access or modify data
- **Backup and Recovery** – Protects data from loss or corruption
- **Data Management** – Organizes and maintains database structure

Examples of DBMS

- MySQL
- PostgreSQL
- Oracle Database
- SQLite

What is SQL?

SQL (Structured Query Language) is the standard programming language used to **communicate with databases**.

It allows users to **create, retrieve, update, and delete data** stored in a database.

Common SQL Operations

Operation	Purpose
INSERT	Add new records to a table
SELECT	Retrieve data from a database
UPDATE	Modify existing data
DELETE	Remove records from a table

These four operations together are called **CRUD Operations**.

CRUD Meaning

CRUD stands for:

- **Create** → Add new data
- **Read** → Retrieve existing data
- **Update** → Modify stored data
- **Delete** → Remove data from database

Example

If a database stores student records:

- **INSERT** → Add a new student
- **SELECT** → View student details
- **UPDATE** → Change student email
- **DELETE** → Remove a student record

What is MySQL?

MySQL is an **open-source Relational Database Management System (RDBMS)** that uses SQL for managing databases.

It is one of the **most widely used database systems in web development** because of its speed, reliability, and ease of use.

Key Features of MySQL

- Open-source and free to use
- High performance and reliability
- Supports large databases
- Works well with web technologies like **PHP and Node.js**

Platforms That Use MySQL

- WordPress
- Facebook (in its early days)
- YouTube
- Many e-commerce websites

Relational vs Non-Relational Databases

Databases can be broadly divided into **Relational (SQL)** and **Non-relational (NoSQL)** databases.

Relational Databases (SQL)

Relational databases store data in **tables consisting of rows and columns**.

- Each table has a **fixed structure called a schema**.
- Tables can be connected using **relationships**.

Examples

- MySQL
- PostgreSQL
- Oracle Database

Best For

- Structured data
- Complex queries
- Financial systems
- Enterprise applications

Non-Relational Databases (NoSQL)

Non-relational databases store data in **flexible formats** such as:

- Documents
- Key-value pairs
- Graphs

They **do not require a fixed schema**, making them more flexible for handling unstructured data.

Examples

- MongoDB
- Firebase
- Cassandra

Best For

- Large-scale applications
- Real-time systems
- Unstructured or semi-structured data

Creating a Database in MySQL

Working with MySQL begins with creating a database. A database acts as a container that stores related tables and the data inside them. Once a database is created, tables can be added to organize and manage information efficiently.

Creating a Database

To start using MySQL, a new database must first be created. The command used for this purpose defines the name of the database that will store tables and records.

Syntax

CREATE DATABASE database_name;

Example

```
CREATE DATABASE student_db;
```

This command creates a database named **student_db** where tables related to students can be stored.

Good Practices for Naming Databases

- Database names should always be **unique**.
- Avoid using **spaces or special characters**.
- Use **lowercase letters and underscores** for better readability.

Example of a clear and readable name:

employee_records

Viewing All Databases

After creating databases, it is often useful to see the list of all databases available in the MySQL server.

Command

SHOW DATABASES;

This command displays every database that exists in the MySQL system.

Switching to a Database

Before creating tables or inserting data, the **desired database** must be selected. This tells MySQL where the tables and records should be stored.

Command

USE student_db;

After executing this command, all further operations will be performed inside the **student_db** database.

Dropping a Database

Sometimes a database is no longer required. In such cases, it can be deleted using the DROP command.

Syntax

DROP DATABASE database_name;

Example

DROP DATABASE student_db;

This command permanently deletes the database along with all its tables and data.

Important note

- This operation **cannot be undone**.
- Always double-check before deleting a database.

Creating a Table in MySQL

Once a database is selected, the next step is creating tables. Tables are used to organize data into rows and columns.

Each table stores data about a specific entity such as students, employees, products, or orders.

Table Creation Syntax

```
CREATE TABLE table_name (  
    column1 datatype constraints,  
    column2 datatype constraints  
);
```

Example Table

```
CREATE TABLE students (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL DEFAULT 'No Name',  
    age INT,  
    email VARCHAR(100) UNIQUE,  
    admission_date DATE  
);
```

Explanation of Columns

id

- Data type: INT
- Automatically increases for every new record
- Acts as the primary key that uniquely identifies each student

name

- Data type: VARCHAR
- Maximum length of characters allowed
- Cannot be empty because of the NOT NULL constraint
- Default value is set to "No Name"

age

- Data type: INT

- Stores numerical values representing student age

email

- Data type: VARCHAR
- Must contain unique values for each student

admission_date

- Data type: DATE
- Stores the admission date of the student

Commonly Used Data Types

Data types define the type of data that can be stored in a column.

INT

- Stores whole numbers
- Example uses include age, quantity, and counts

VARCHAR

- Stores variable length text
- Commonly used for names, emails, and titles

TEXT

- Stores long pieces of text such as descriptions or comments

DATE

- Stores date values in the format year-month-day

DATETIME

- Stores both date and time information

BOOLEAN

- Stores logical values such as TRUE or FALSE

Common Constraints

Constraints are rules applied to table columns to maintain data accuracy and integrity.

PRIMARY KEY

- Uniquely identifies each record in a table

NOT NULL

- Prevents a column from storing empty values

UNIQUE

- Ensures that all values in a column are different

AUTO_INCREMENT

- Automatically increases the value of a numeric column when new records are inserted

DEFAULT

- Assigns a default value if no value is provided

FOREIGN KEY

- Creates relationships between tables

Viewing Tables and Table Information

After creating tables, several commands help inspect the database structure and stored data.

View all tables in the current database

SHOW TABLES;

View the structure of a table

DESCRIBE students;

This command displays information about columns, data types, and constraints.

View data stored in a table

SELECT * FROM students;

This command retrieves all records stored in the students table.

Tables form the **core structure of relational databases**. Properly designed tables help maintain organized data and prevent issues such as redundancy and inconsistency.

Modifying a Table in MySQL

As applications grow and requirements change, it becomes necessary to modify existing tables. MySQL provides several commands to update table structure without deleting the table.

These modifications are performed using **ALTER TABLE** and related commands.

Renaming a Table

If the name of a table needs to be changed, the RENAME TABLE command can be used.

RENAME TABLE old_table_name TO new_table_name;

This changes the table name while keeping the data intact.

Dropping a Table

If a table is no longer needed, it can be permanently removed.

DROP TABLE table_name;

This deletes the table along with all records stored in it.

Renaming a Column

If a column name needs to be changed, the ALTER TABLE command can be used.

ALTER TABLE table_name

RENAME COLUMN old_column_name TO new_column_name;

This updates the column name without affecting stored data.

Dropping a Column

To remove an existing column from a table:

ALTER TABLE table_name

DROP COLUMN column_name;

This deletes the column and all values stored in it.

Adding a Column

A new column can be added to an existing table when additional information needs to be stored.

ALTER TABLE table_name

ADD COLUMN column_name datatype constraints;

Example:

```
ALTER TABLE students
```

```
ADD COLUMN gender VARCHAR(10);
```

This adds a new column called **gender** to the students table.

Modifying a Column

If the data type or constraints of a column need to be changed, the MODIFY command can be used.

ALTER TABLE table_name

MODIFY COLUMN column_name new_datatype new_constraints;

Example:

```
ALTER TABLE students
```

```
MODIFY COLUMN name VARCHAR(150) NOT NULL;
```

This increases the length of the name column and ensures that it cannot be empty.

Changing the Order of Columns

Sometimes it may be necessary to change the order of columns in a table. This can be done using the AFTER keyword.

ALTER TABLE table_name

MODIFY COLUMN column_name datatype AFTER another_column_name;

This moves the column to appear after the specified column.

Inserting Rows into a Table in MySQL

In MySQL, data is added to a table using the **INSERT INTO** statement. This command allows users to insert records into a table either **one row at a time** or **multiple rows in a single query**.

Before inserting data, a database must exist and a table must be created inside that database.

Data can be inserted into a table using the **INSERT INTO** statement. This statement specifies the table name, column names, and the values to be stored.

There are two common approaches to inserting records.

Inserting One Row at a Time

Each record can be inserted individually using separate INSERT statements.

Example:

```
INSERT INTO student (id, name, age, grade, date_of_birth)
VALUES (1, 'Ayesha Khan', 16, '10th', '2007-05-15');
```

```
INSERT INTO student (id, name, age, grade, date_of_birth)
VALUES (2, 'Ravi Sharma', 17, '11th', '2006-03-22');
```

```
INSERT INTO student (id, name, age, grade, date_of_birth)
VALUES (3, 'Meena Joshi', 15, '9th', NULL);
```

```
INSERT INTO student (id, name, age, grade, date_of_birth)
VALUES (4, 'Arjun Verma', 18, '12th', NULL);
```

```
INSERT INTO student (id, name, age, grade, date_of_birth)
VALUES (5, 'Sara Ali', 16, '10th', NULL);
```

Understanding NULL Values

- **NULL** represents missing or unknown data.
- In the example above, the **date_of_birth** field is not provided for some students, so it is stored as NULL.

Inserting Multiple Rows at Once

Instead of running multiple INSERT commands, several records can be inserted in a single statement. This method is more efficient when inserting large amounts of data.

Example:

```
INSERT INTO student (id, name, age, grade) VALUES
```

```
(15, 'Ayesha Khan', 16, '10th'),
```

```
(25, 'Ravi Sharma', 17, '11th'),
```

```
(35, 'Meena Joshi', 15, '9th'),
```

```
(45, 'Arjun Verma', 18, '12th'),
```

```
(55, 'Sara Ali', 16, '10th'),
```

```
(65, 'Karan Mehta', 17, '11th'),
```

```
(75, 'Tanya Roy', 15, '9th'),
```

```
(85, 'Vikram Singh', 18, '12th'),
```

```
(95, 'Anjali Desai', 16, '10th'),
```

```
(105, 'Farhan Zaidi', 17, '11th');
```

Advantages of Multiple Row Insert

- Reduces the number of queries executed
- Improves performance when inserting many records
- Keeps code cleaner and easier to maintain

Selecting Data in MySQL

The **SELECT** statement is used to retrieve data from a table in MySQL. It allows users to view all records, specific columns, or filtered results based on certain conditions.

The SELECT command becomes very powerful when combined with filtering conditions, comparison operators, sorting, and pattern matching.

Basic SELECT Statement

The simplest way to retrieve data from a table is by using the SELECT statement.

To retrieve all data from the **student** table:

SELECT * FROM student;

The asterisk symbol represents **all columns** in the table. This command returns every row and column stored in the table.

To retrieve only specific columns:

SELECT name, grade FROM student;

This query displays only the **name** and **grade** columns instead of the entire table.

Using specific columns is helpful when working with large tables because it reduces unnecessary data retrieval.

Using the WHERE Clause

The **WHERE** clause is used to filter rows based on specific conditions. It allows users to retrieve only the records that match certain criteria.

Example: selecting students who are in the tenth grade.

SELECT * FROM student

WHERE grade = '10th';

Example: selecting students older than sixteen.

SELECT * FROM student

WHERE age > 16;

The WHERE clause can be used with different comparison operators to refine the results.

Comparison Operators in MySQL

Comparison operators are used inside the WHERE clause to compare values.

Operator	Description	Example
=	Equal to	WHERE age = 16
!=	Not equal to	WHERE grade != '12th'
<>	Alternative for not equal	WHERE grade <> '12th'
>	Greater than	WHERE age > 16
<	Less than	WHERE age < 17
>=	Greater than or equal to	WHERE age >= 16
<=	Less than or equal to	WHERE age <= 18
BETWEEN	Within a range	WHERE age BETWEEN 15 AND 17
IN	Matches any value in a list	WHERE grade IN ('10th','12th')
NOT IN	Excludes values in a list	WHERE grade NOT IN ('9th','11th')
LIKE	Pattern matching	WHERE name LIKE 'A%'
NOT LIKE	Pattern not matching	WHERE name NOT LIKE '%a'

Example using a comparison operator:

SELECT * FROM student

WHERE age > 16;

This query returns students whose age is greater than sixteen.

Handling NULL Values

NULL represents **missing or unknown data** in a database.

It is important to understand that NULL is not equal to:

- zero
- an empty string
- any other value

Because of this, NULL must be handled using special operators.

Common mistake

The following query will not work correctly:

```
SELECT * FROM student
```

```
WHERE grade = NULL;
```

Correct ways to check NULL values

Condition	Correct Syntax
Check for NULL values	WHERE grade IS NULL
Check for non-NULL values	WHERE grade IS NOT NULL

Example: selecting students with no grade assigned.

```
SELECT * FROM student
```

```
WHERE grade IS NULL;
```

Example: selecting students who already have a grade.

```
SELECT * FROM student
```

```
WHERE grade IS NOT NULL;
```

Combining Conditions

Multiple conditions can be combined using logical operators.

Common logical operators include:

- AND
- OR
- parentheses for grouping conditions

Example: students in tenth grade and older than sixteen.

```
SELECT * FROM student
```

```
WHERE grade = '10th' AND age > 16;
```

Example: students in ninth or twelfth grade.

```
SELECT * FROM student
```

```
WHERE grade = '9th' OR grade = '12th';
```

Example with multiple grouped conditions:

SELECT * FROM student

WHERE (grade = '10th' OR grade = '11th') AND age >= 16;

Parentheses help control the order in which conditions are evaluated.

Sorting Results with ORDER BY

The **ORDER BY** clause is used to sort query results based on one or more columns.

Example: sorting students by age in ascending order.

SELECT * FROM student

ORDER BY age ASC;

Example: sorting students by name in descending order.

SELECT * FROM student

ORDER BY name DESC;

Ascending order sorts from smallest to largest, while descending order sorts from largest to smallest.

Limiting Results with LIMIT

The **LIMIT** clause restricts the number of rows returned by a query.

Example: retrieving only five rows.

SELECT * FROM student

LIMIT 5;

Example: retrieving five rows starting after the first two rows.

SELECT * FROM student

LIMIT 2, 5;

The first value represents the **offset** and the second value represents the **number of rows to return**.

Using Wildcards with LIKE

Wildcards are used with the **LIKE** operator to search for patterns in text.

They are useful when the exact value is not known.

There are two main wildcards used in MySQL.

- % - percent symbol matches zero or more characters
- _ - underscore matches exactly one character

Example: names starting with the letter A.

SELECT * FROM student

WHERE name LIKE 'A%';

This query finds names such as Aakash, Ananya, or Aryan.

Matching Dates Using the Underscore Wildcard

The underscore wildcard is useful for matching specific positions in a pattern, especially when working with date values.

Example: selecting records where the day is the fifth of any month.

SELECT * FROM attendance

WHERE date LIKE '____-__-05';

Explanation of the pattern:

- four underscores represent any four-digit year
- two underscores represent any two-digit month
- zero five represents the fifth day

This query retrieves records where the date ends with **dash zero five**, meaning the fifth day of any month in any year.

Examples of Date Pattern Matching

Pattern	Matches
'2025-05-%'	Any date in May twenty twenty five
'2024-12-__'	Any day in December twenty twenty four
'____-01-01'	First January of any year
'202_- -'	Any date in the twenty twenties decade
'__-__-3'	Days from thirty to thirty nine

Some patterns may match syntactically even if they represent invalid calendar dates.

LIKE Patterns

Pattern	Meaning
'A%'	Starts with the letter A
'%sh'	Ends with the letters sh
'%ar%'	Contains the letters ar anywhere
'R_____'	Five-letter word starting with R
'____-__-05'	Dates where the day is the fifth

Updating Data in a MySQL Table

In MySQL, existing records in a table can be modified using the **UPDATE** statement. This command allows changes to be made to one row, multiple rows, or even every row in a table.

The UPDATE statement is commonly used when correcting information, modifying records, or updating values based on certain conditions.

Basic Syntax of UPDATE

The UPDATE statement modifies values stored in a table.

UPDATE table_name

SET column1 = value1, column2 = value2

WHERE condition;

Explanation of the main components:

- **UPDATE** specifies the table that will be modified
- **SET** assigns new values to one or more columns
- **WHERE** determines which rows will be affected

The WHERE clause is extremely important because it prevents accidental updates to all rows in the table.

Example Table

Assume the following **student** table contains these records:

id	name	age	grade
1	Ayesha Khan	16	10th
2	Ravi Sharma	17	11th
3	Meena Joshi	15	9th

This table stores information about students including their ID, name, age, and grade.

Updating a Single Row

Sometimes only one specific record needs to be changed. This is done by targeting the row using a condition in the WHERE clause.

Example: changing the grade of the student whose ID is two.

```
UPDATE student  
SET grade = '12th'  
WHERE id = 2;
```

This query updates only the row where the student ID matches the specified value.

Updating Multiple Columns

More than one column can be updated in a single statement by separating assignments with commas.

Example: updating both the age and grade of the student with ID three.

```
UPDATE student  
SET age = 17, grade = '10th'  
WHERE id = 3;
```

This query changes the student's age and grade at the same time.

Updating All Rows

If the WHERE clause is omitted, the UPDATE statement will modify every row in the table.

Example: setting the age of all students to eighteen.

UPDATE student

SET age = 18;

Important warning:

Running an UPDATE statement without a WHERE condition affects the entire table and may overwrite existing data unintentionally.

Conditional Updates Using Comparison Operators

The WHERE clause can include comparison operators to update rows that match certain conditions.

Example: promoting all students currently in the ninth grade to the tenth grade.

UPDATE student

SET grade = '10th'

WHERE grade = '9th';

Example: increasing the age of students who are younger than eighteen.

UPDATE student

SET age = age + 1

WHERE age < 18;

In this case, the query increments the age value rather than replacing it.

Updating Values That Are NULL

Sometimes certain columns contain NULL values, meaning the data is missing or unknown. These values can also be updated.

Example: assigning the grade "Unknown" to students whose grade value is NULL.

UPDATE student

SET grade = 'Unknown'

WHERE grade IS NULL;

The **IS NULL** condition is required because NULL cannot be compared using the equal operator.

Deleting Data in a MySQL Table

In MySQL, records can be removed from a table using the **DELETE** statement. This command allows users to delete specific rows, remove rows based on conditions, or delete all records from a table.

Deleting data should always be done carefully because once rows are removed, they cannot be easily recovered unless backups exist.

Basic Syntax of DELETE

The DELETE statement removes rows from a specified table.

DELETE FROM table_name

WHERE condition;

Explanation of the main components:

- **DELETE FROM** specifies the table from which rows will be removed
- **WHERE** determines which rows should be deleted

Important note:

If the WHERE clause is not included, **all rows in the table will be deleted.**

Example Table

Assume the following **student** table contains these records:

id	name	age	grade
1	Ayesha Khan	16	10th
2	Ravi Sharma	17	12th
3	Meena Joshi	15	9th

This table stores information about students including their ID, name, age, and grade.

Deleting a Specific Row

A single row can be removed by specifying a condition that identifies the record.

Example: deleting the student with ID equal to two.

DELETE FROM student

WHERE id = 2;

This query removes only the row where the ID matches the specified value.

Deleting Rows Based on a Condition

Multiple rows can be deleted if they meet a particular condition.

Example: deleting all students who are in the ninth grade.

DELETE FROM student

WHERE grade = '9th';

All rows where the grade is ninth will be removed from the table.

Deleting Rows Using Comparison Operators

Comparison operators can also be used to delete rows based on numeric or logical conditions.

Example: deleting all students younger than sixteen.

DELETE FROM student

WHERE age < 16;

This query removes all rows where the age value is less than sixteen.

Deleting Rows with NULL Values

Rows containing missing values can also be removed.

Example: deleting students whose grade value is NULL.

DELETE FROM student

WHERE grade IS NULL;

The **IS NULL** condition is required because NULL cannot be compared using the equal operator.

Deleting All Rows from a Table

All rows in a table can be removed if the WHERE clause is omitted.

Example: deleting every record from the student table.

DELETE FROM student;

Important note:

This command removes **all data** from the table but **keeps the table structure intact**.

This means the table still exists and new records can still be inserted later.

Completely Removing a Table

If both the data and the table structure need to be deleted, the DROP TABLE command is used.

Example:

DROP TABLE student;

This command permanently removes the entire table along with all of its data.

